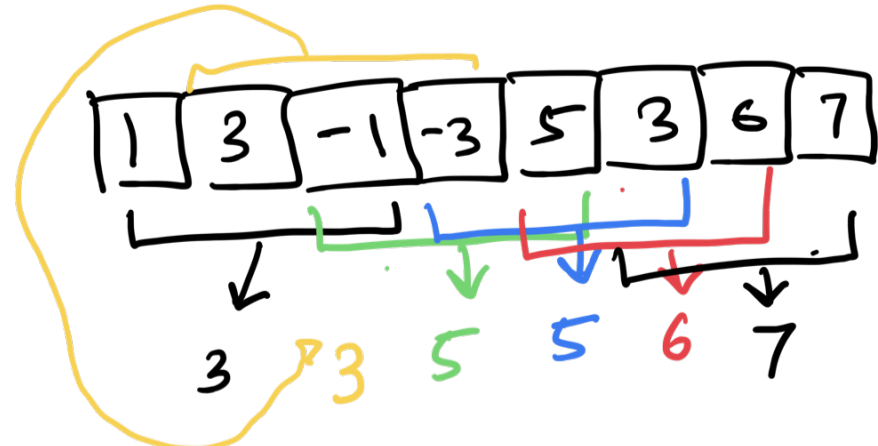


# sliding window maximum

nums = [1, 3, -1, -3, 5, 3, 6, 7] ↓  
max of all subarray  
size k

k = 3

↳ window size



ans = [3, 3, 5, 5, 6, 7]

Brute force

for (i = 0 to n)

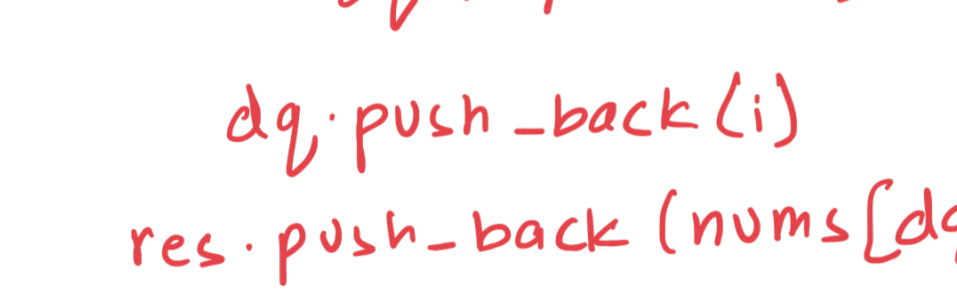
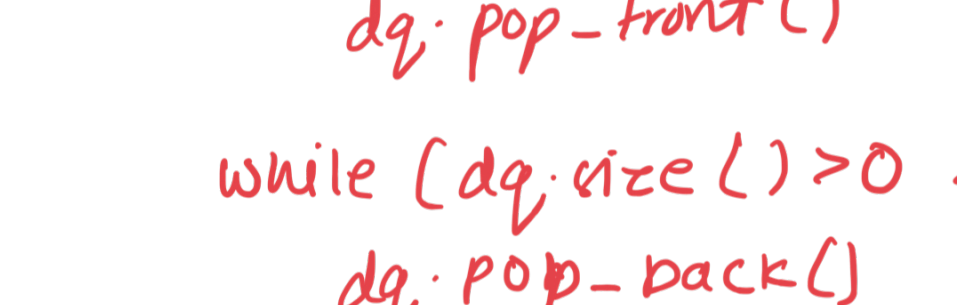
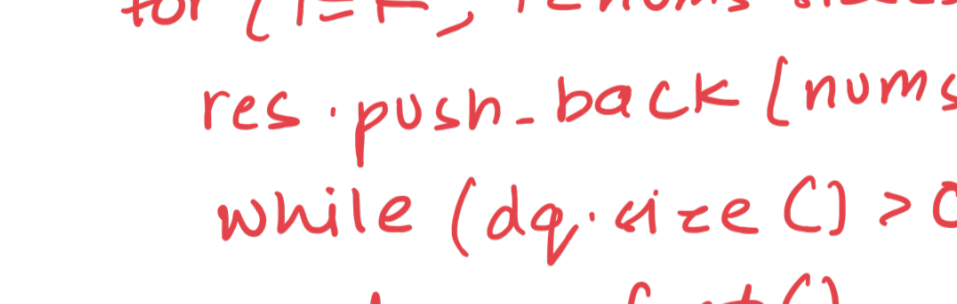
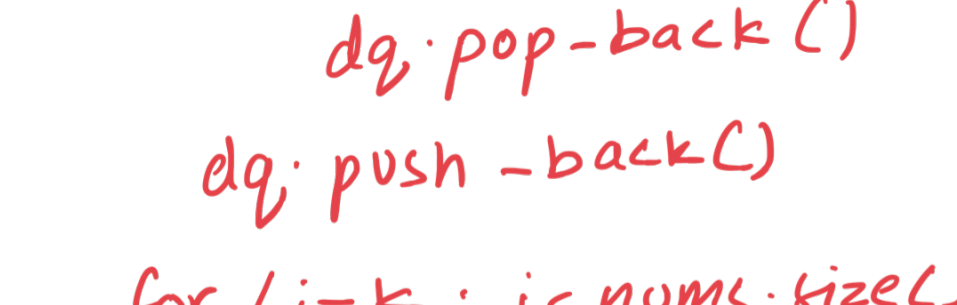
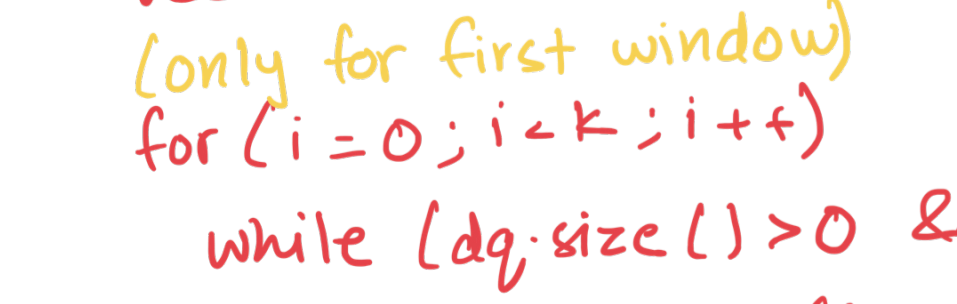
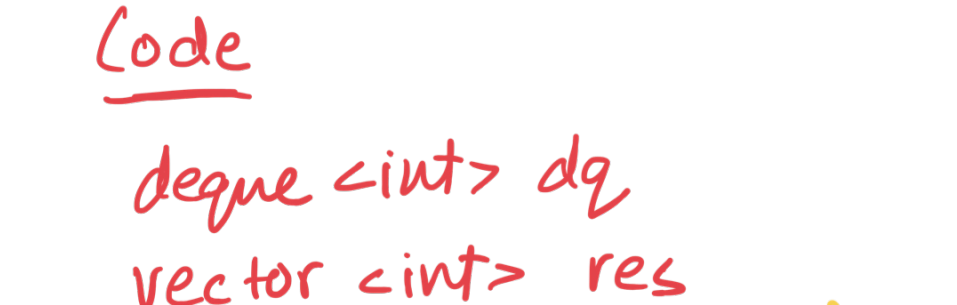
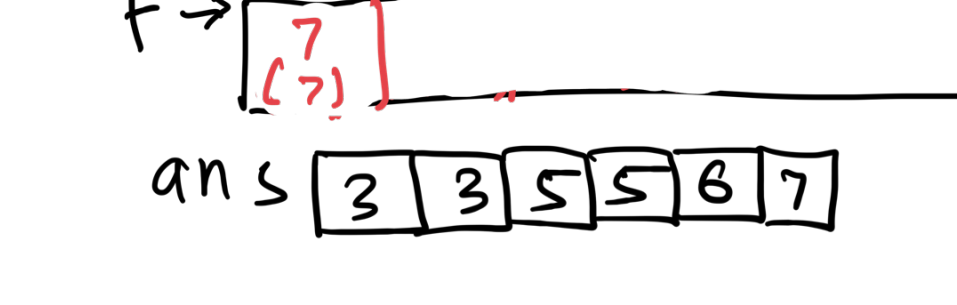
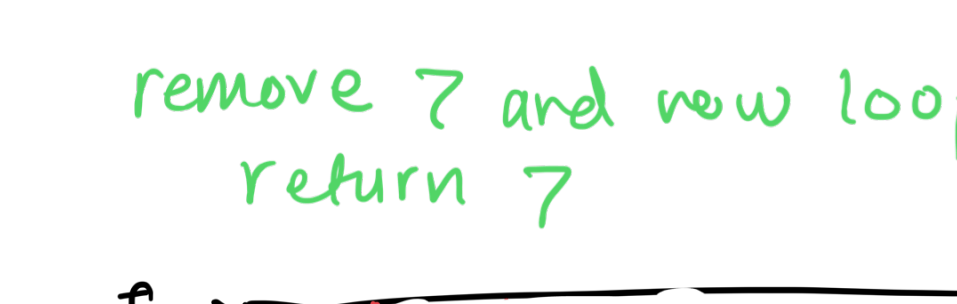
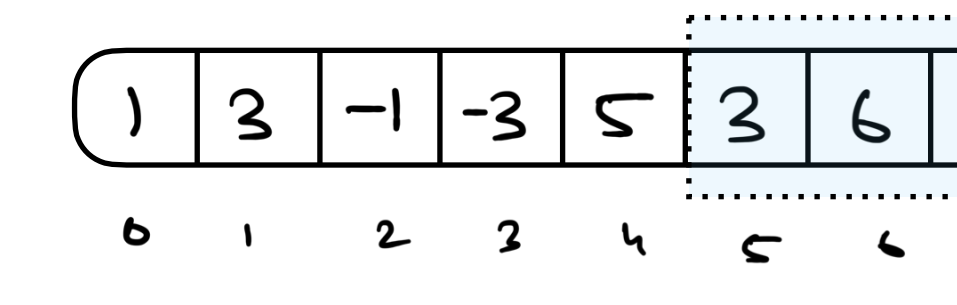
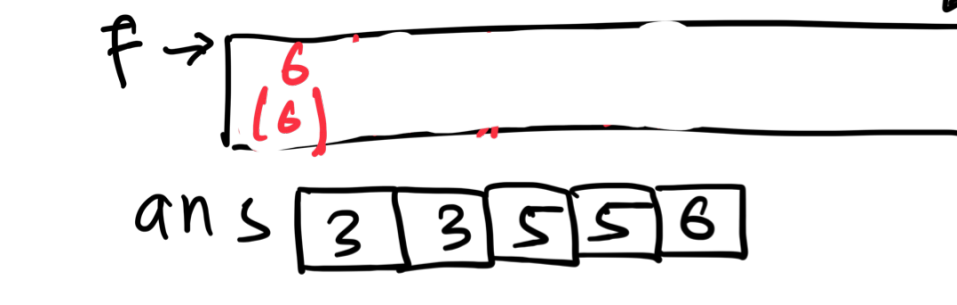
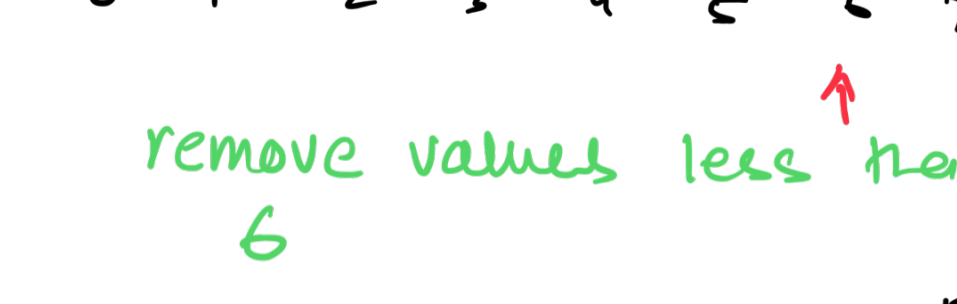
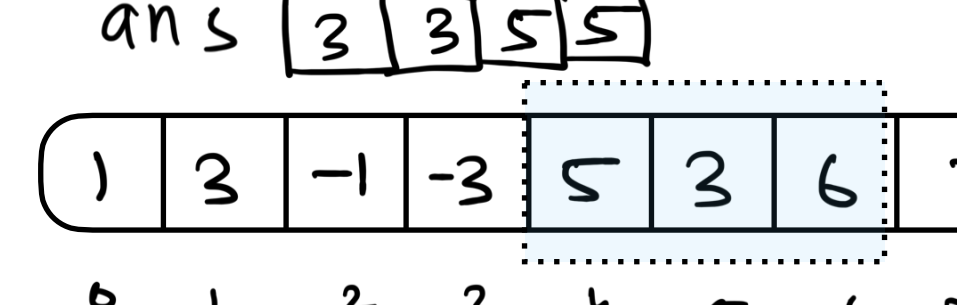
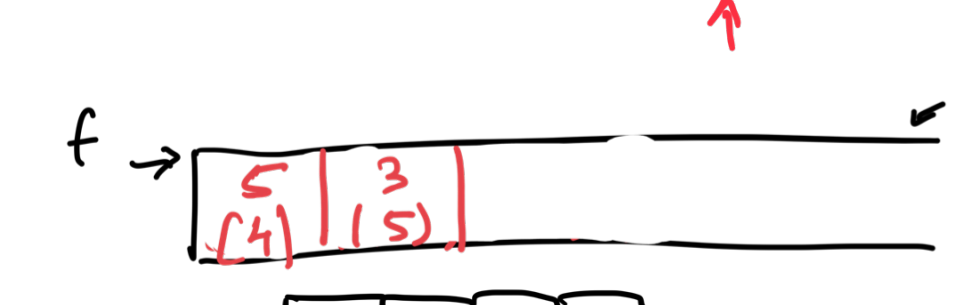
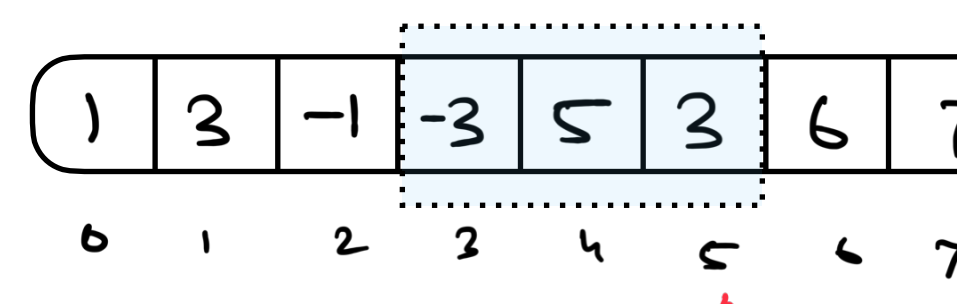
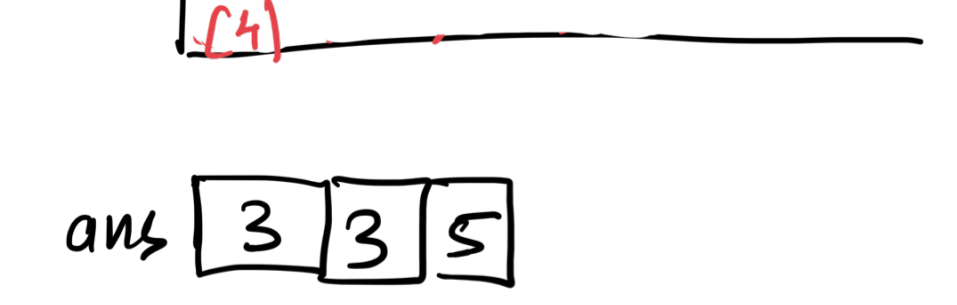
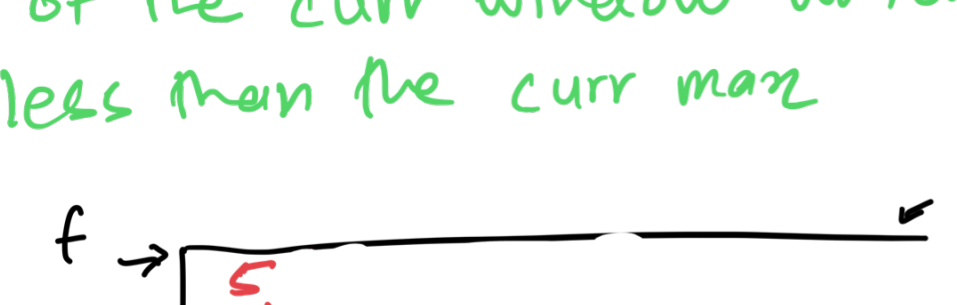
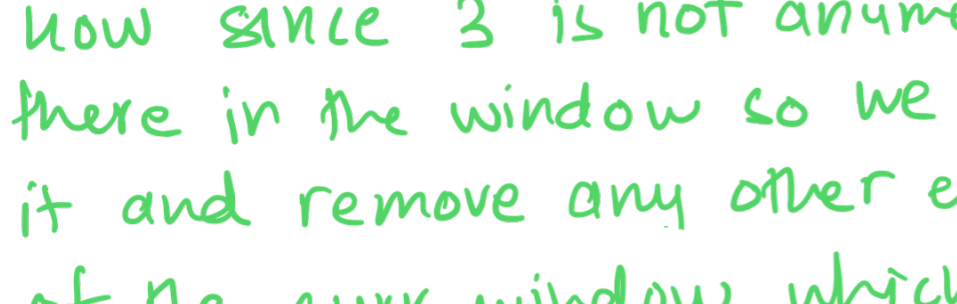
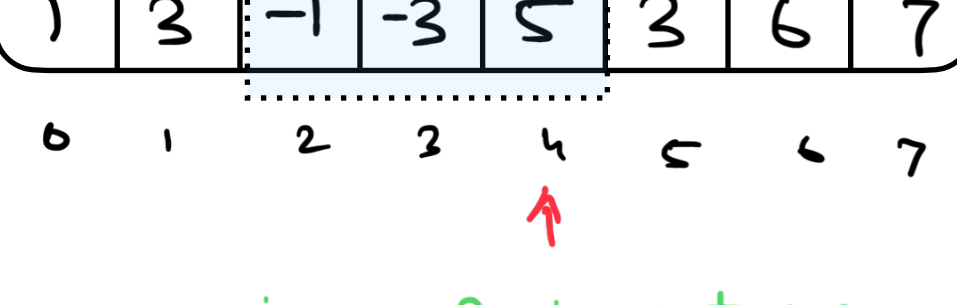
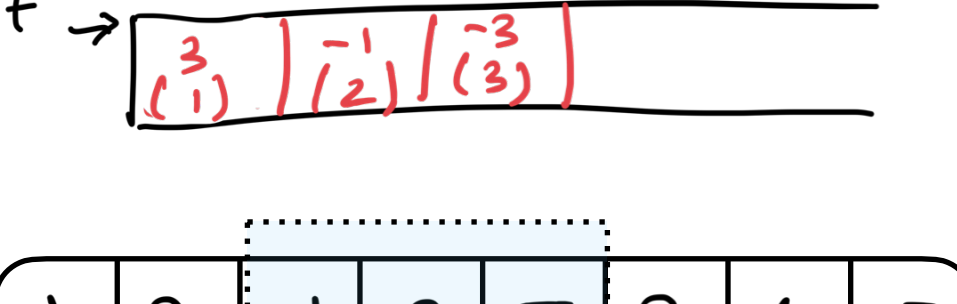
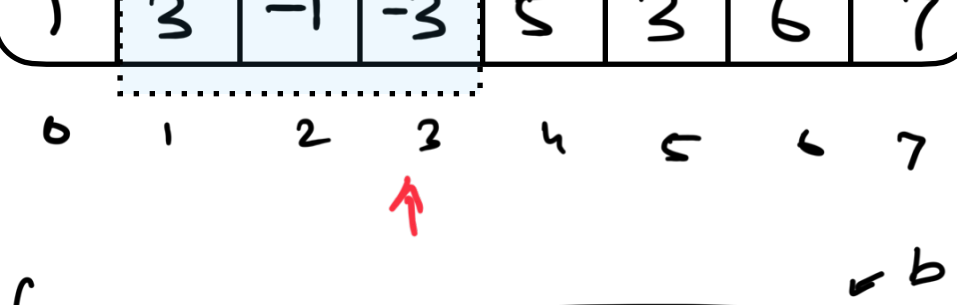
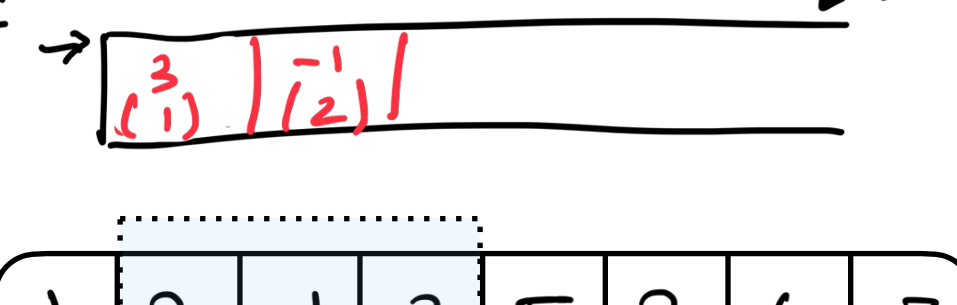
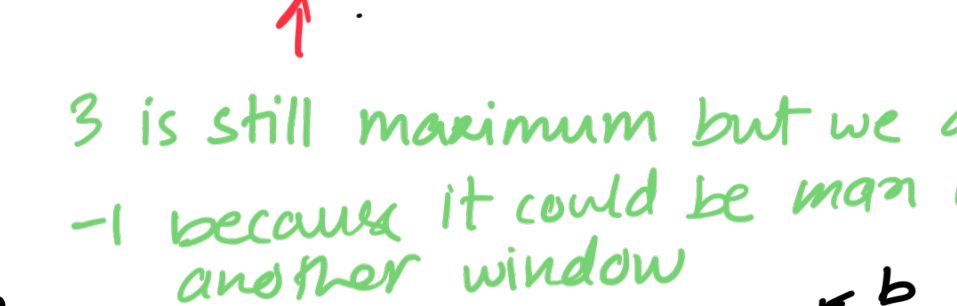
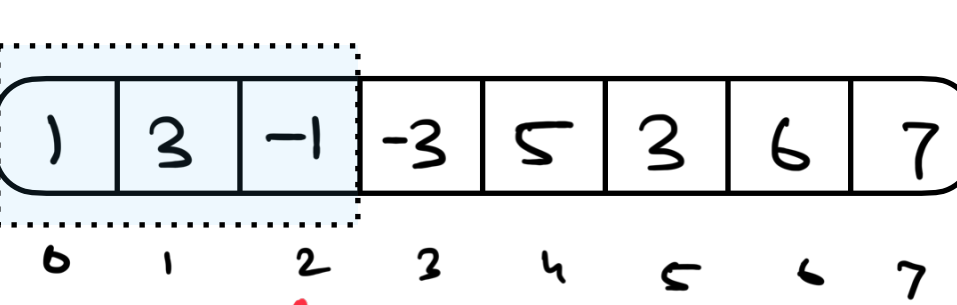
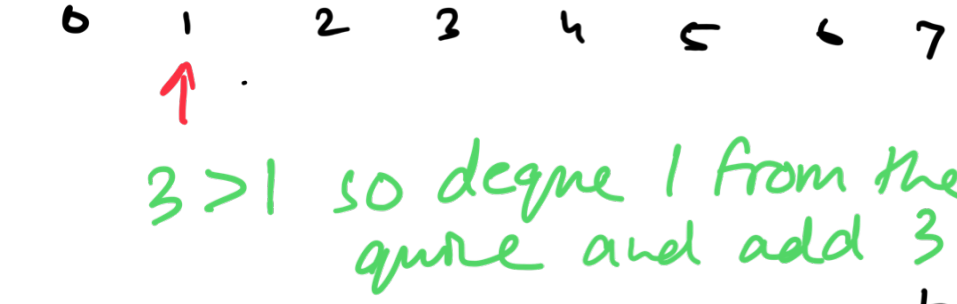
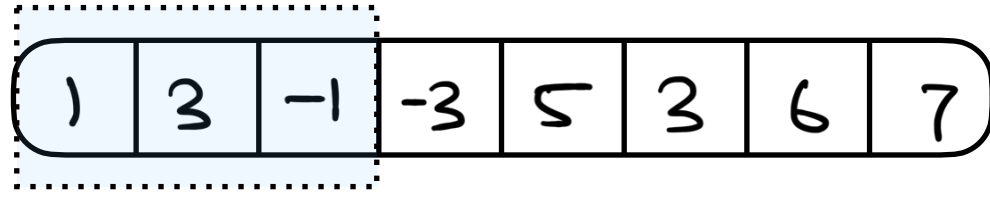
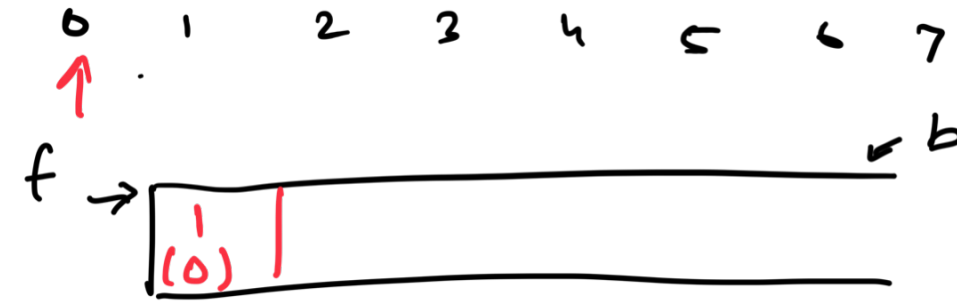
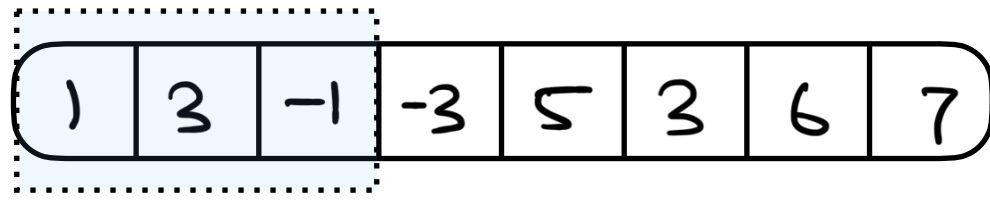
for (j = i to i+k)

add max

TC:  $O(n \cdot k)$

Optimal ⇒ Dequeue data structure

TC:  $O(n)$



## Code

deque<int> dq;

vector<int> res;

(only for first window)

for (i = 0; i < k; i++)

while (dq.size() > 0 && nums[dq.back()] <= nums[i])

if number in queue is less

than current number

remove it

dq.pop-back()

dq.push-back(i)

for (i = k; i < nums.size(); i++)

res.push-back(nums[dq.front()])

add in case it could be

maximum later on

while (dq.size() > 0 && dq.front() <= i - k)

if not part of the

window nital de

dq.pop-front()

while (dq.size() > 0 && nums[dq.back()] <= nums[i])

remove the smaller values

not needed

dq.pop-back()

dq.push-back(i)

res.push-back(nums[dq.front()]);

return res